# OBJECT ORIENTED PROGRAMMING WITH C++

# ASSIGNMENT 1

Design, Implement and Validate the Fare Logic
of a City Metro Smart-Card System

CO1: Apply object-oriented programming principles and C++ constructs

Department of Computer Science & Engineering

Submitted by:

**Rashmi Naurene**

Register Number: 192521357

Under the Guidance of

**Jegedeesh K**

Academic Year 2025–26

Total Marks: 50 | Duration: 1 Week

# INDEX

Task 1 – Understand the Problem and Identify the Objects ..... 4 Marks

Task 2 – Use Tokens, Keywords, Identifiers and Constants Correctly ..... 3 Marks

Task 3 – Choose Data Types and Check Type Compatibility ..... 4 Marks

Task 4 – Control Implicit Conversions ..... 4 Marks

Task 5 – Write Correct Expressions with the Right Precedence ..... 4 Marks

Task 6 – Design the Control Structures ..... 4 Marks

Task 7 – From Procedural to Object-Oriented ..... 4 Marks

Task 8 – Read and Print Data Safely ..... 4 Marks

Task 9 – Implement and Test the Fare Logic ..... 4 Marks

Task 10 – Compare Expectations with Execution ..... 4 Marks

Task 11 – Recommend and Defend the Final Design ..... 4 Marks

Task 12 – Day-4 Design Challenge (Re-evaluate and Adapt) ..... 8 Marks

Total: 50 Marks

# TASK 1 – Understand the Problem and Identify the Objects

## Object-Oriented Analysis of the Metro Fare System

### 1.1 Objects in the Problem

From an object-oriented viewpoint the metro fare system contains the following key objects. Each object is justified by the data it owns and the behaviour it must provide.

**SmartCard**

Data: balance (in paise), cardId, cardType (regular / student / senior). Behaviour: canTravel(fare), deduct(fare), recharge(amount), getBalance(). Justification: The balance must never become negative and at least Rs 50 must remain after every successful journey. Encapsulating the balance inside the card guarantees that no external code can violate this rule.

**Passenger**

Data: name, age, category (student / senior / regular). Behaviour: isEligibleForConcession(). Justification: Concession rules depend on the passenger category; keeping the category with the passenger object makes the rule local and easy to change.

**Journey / Trip**

Data: entryZone, exitZone, hour, isWeekend, fareCharged. Behaviour: computeFare(), isPeak(). Justification: A journey is the unit of work that produces a fare; all inputs needed for the calculation belong together.

**Zone**

Data: zoneId, zoneName, baseFarePaise. Behaviour: getBaseFare(). Justification: Zones are discrete values with associated fares; modelling them as an enum class (or later a class) prevents invalid zone numbers.

**FareTable**

Data: base fares for each zone, peak surcharge percent, concession rates, minimum balance. Behaviour: lookupFare(zone, peak, concession). Justification: All monetary constants live in one place so a fare revision requires only one change.

### 1.2 Procedural vs Object-Oriented Style

In a purely procedural style the same problem would be written with global variables (balance, currentZone, isPeak) and free functions (computeFare, canTravel, deduct). Any function could read or write the balance, making it easy to produce a negative balance by accident. When a fare rule changes, every place that hard-codes the number must be found and edited.

In an object-oriented style the balance is private inside SmartCard. Only the member functions deduct() and canTravel() may change or inspect it, so the “balance never below Rs 50” rule is enforced in one place. Adding a new zone or a new discount becomes a local change inside FareTable or a new method, rather than a search-and-replace across the whole program.

## 1.3 Basic Principles of OOP Applied to the System

- Encapsulation – SmartCard hides its balance; clients call deduct() and canTravel() instead of touching the variable directly.
- Abstraction – The gate software only needs to know "can this card travel for this fare?"; the internal representation in paise is hidden.
- Inheritance – A future StudentCard or SeniorCard can inherit from SmartCard and override the concession rate, or a MonthlyPass can inherit from a Ticket base class.
- Polymorphism – Different card types can respond to the same deduct() call with different concession logic through virtual functions.

## 1.4 Rules Every Correct Journey Must Satisfy

1. After a successful journey the card balance must be at least Rs 50 (5000 paise).
2. The fare must be calculated from the farthest zone travelled, using integer paise only.
3. Peak surcharge (20 %) applies only for hours in [8,10) and [17,20); 10:00 and 20:00 are off-peak.
4. A journey is refused if the resulting balance would fall below the minimum.
5. Concession (student / senior) is applied only when the passenger category qualifies.

Assumptions: Zones are ordered Inner < Middle < Outer; the farthest zone determines the fare; time is given as an integer hour 0–23; money is always stored as long long paise.

## 1.5 Alternative Concept – Purely Procedural Style

If the system were written with global variables and free functions, every function would receive the balance by reference or would access a global. Changing the minimum-balance rule would require editing every function that performs a deduction. Time and space per journey remain O(1), but the amount of code that must be changed when one rule changes grows linearly with the number of call sites, whereas the class design confines the change to a single method.

# TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly

## 2.1 Five Kinds of Tokens with Examples

- Keywords – int, long, bool, constexpr, enum, class, if, switch, return, true, false
- Identifiers – balancePaise, BASE_PAISE, canTravel, Zone, SmartCard, fare
- Constants – 1000, 2000, 3000, 20, 5000, 'A', true
- Operators – +, -, *, /, %, >=, &&, ||, =, -=, ?: , ::
- Punctuators – { } ( ) [ ] ; , .

## 2.2 #define vs const vs constexpr

#define creates a textual macro with no type and no scope; it is expanded by the preprocessor and can produce surprising results when used with operators. const creates a typed, scoped variable that may still occupy storage if its address is taken. constexpr guarantees a compile-time constant expression that can be used in array sizes, case labels and template arguments, and usually occupies no storage at run time.

For the fare table constexpr is preferred because the values are known at compile time, they participate in constant expressions, and the compiler can optimise them aggressively while still performing type checking.

## 2.3 enum vs enum class for Zones

A plain enum pollutes the enclosing scope and allows implicit conversion to int, so Zone::Inner + 1 is legal and may produce an invalid zone. enum class (scoped enumeration) keeps the enumerators inside the enum's scope and forbids implicit conversion to int, forcing an explicit static_cast when an integer is really needed. This safety is essential for a zone value that must never be out of range.

## 2.4 Recommended Constants

```
enum class Zone { Inner = 0, Middle = 1, Outer = 2 };

constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, 20, 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;   // Rs 50
constexpr int STUDENT_CONCESSION_PERCENT = 25;
constexpr int SENIOR_CONCESSION_PERCENT  = 30;
```

## 2.5 Alternative Concept – #define Macros

If fares were written with #define BASE_INNER 1000 the symbols would have no type, no scope and no debugger visibility. A typo such as BASE_INNER +1 would still compile. Memory usage is essentially the same (neither a macro nor a constexpr value needs run-time storage), and running time is identical, but the lack of type checking makes the macro version far more error-prone when the fare table grows.

# TASK 3 – Choose Data Types and Check Type Compatibility

## 3.1 Data-Type Table

| Value | Chosen Type | sizeof (typical) | Reason |
|---|---|---|---|
| Fare / balance | long long | 8 bytes | Holds network total safely; no floating error |
| Zone index | enum class / int | 4 bytes | Small discrete set |
| Hour (0–23) | int | 4 bytes | Simple arithmetic, no overflow risk |
| Peak flag | bool | 1 byte | True/false only |
| Journey count | long long | 8 bytes | 3 000 000 journeys need > 32-bit range |
| Card ID | std::string / long | var / 8 | Unique identifier |
| Concession % | int | 4 bytes | Small integer percentage |

## 3.2 Why Money Must Be Stored in Integer Paise

Floating-point types (float, double) cannot represent most decimal fractions exactly. Adding Rs 0.10 (0.1) many times accumulates a rounding error that eventually produces a balance that is one paisa short or over. Storing every amount as an integer number of paise (1 Rs = 100 paise) guarantees that every addition and subtraction is exact. All arithmetic stays in the integer domain; conversion to rupees is performed only for display (value / 100 and value % 100).

## 3.3 Type Compatibility and Implicit Conversions

When an int and a double appear in the same expression the int is promoted to double. When a bool is used in arithmetic it becomes 0 or 1. Mixing signed and unsigned integers causes the signed value to be converted to unsigned, which can turn a negative number into a large positive value and reverse the result of a comparison. The compiler warns about many of these conversions when -Wall -Wextra is used.

## 3.4 Ranges and Overflow

From <climits>, INT_MAX is typically 2 147 483 647. A single day's collection at 3 000 000 journeys × Rs 30 = Rs 90 000 000, which still fits in a 32-bit int, but the cumulative total over many days or the sum of all cards does not. Therefore long long is required for any network-wide accumulator.

## 3.5 Alternative Concept – double Rupees

Storing money as double gives the same asymptotic time (O(1) per addition) and the same 8-byte size. However, after only a few thousand additions the representation error becomes visible. Measuring the sum of 20 000 fares of Rs 10.00 stored as double already shows a discrepancy of several paise compared with the exact integer-paise total. At 3 000 000 journeys the error is large enough to affect audit reports.

# TASK 4 – Control Implicit Conversions

## 4.1 Common Conversion Points in the Fare System

- Integer division: a / b truncates toward zero when both operands are int.
- Integer-to-double: needed when a percentage of a fare is calculated; must be written with an explicit cast or by multiplying first.
- Signed/unsigned comparison: a negative int compared with an unsigned value is converted to a large unsigned number.
- Narrowing: assigning a double or a large long long to an int silently loses information unless brace initialisation is used.
- bool conversion: any non-zero integer becomes true.

## 4.2 Predict the Output (written before running)

My predicted output:

```
3 3.5
C 68
1 true
s >= u
```

(Prediction rationale: 7/2 is integer division → 3; static_cast forces floating division → 3.5; 'A'+2 → 'C'; ch+1 promotes to int 68; bool(5) → 1; -1 converted to unsigned becomes a large number, so s < u is false.)

## 4.3 Actual Output and Explanation

Actual output after compilation and execution:

```
3 3.5
C 68
1 true
s >= u
```

Line-by-line conversion analysis:

- c = a/b → integer division of two ints yields 3; the result is then converted to double.
- d = static_cast<double>(a)/b → a is converted to double first, so true floating division occurs.
- ch = 'A'+2 → character arithmetic; 'A' is promoted to int, 65+2=67, stored back as char 'C'.
- ch+1 → char is promoted to int for the arithmetic, printing 68.
- bool flag = 5 → non-zero int converts to true; printed as 1 (default) then as true (boolalpha).
- s < u → s (-1) is converted to unsigned int (UINT_MAX), which is not less than 1, therefore "s >= u".

## 4.4 Corrected Style with Explicit Casts and Brace Initialisation

Using brace initialisation (int x{3.7};) is a compile-time error, catching narrowing at the source. Writing every conversion with static_cast makes the programmer's intention visible and prevents accidental signed/unsigned or integer-division surprises. Running time and memory are unchanged; the only difference is that many classes of error move from run time to compile time.

# TASK 5 – Write Correct Expressions with the Right Precedence

## 5.1 Peak-Hour, Surcharge and Concession Expressions

Peak check (correct and readable):

```
bool isPeak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
```

The parentheses make the two intervals clear. Without them the expression is still correct because && binds more tightly than ||, but it is harder to read and easier to modify incorrectly.

Surcharge (integer arithmetic only):

```
long long fare = base + (base * PEAK_SURCHARGE_PERCENT) / 100;
```

Concession:

```
fare = fare - (fare * concessionPercent) / 100;
```

## 5.2 Operator Precedence Notes

- *, / and % bind more tightly than + and −, and are left-associative.
- Relational operators (<, >= …) bind more tightly than &&, which binds more tightly than ||.
- The conditional operator ?: has lower precedence than most binary operators; its middle operand is treated as a group.
- x++ yields the old value then increments; ++x increments then yields the new value. The expression i = i++ is undefined behaviour in pre-C++17 and should never be written.
- Integer division truncates toward zero; the sign of a % b follows the dividend in modern C++.

## 5.3 Predict the Output (written before running)

My predicted output:

```
11 16
10 6 4
6 4
3 1 -3 -1
18
```

## 5.4 Actual Output and Operator-by-Operator Explanation

Actual output matches the prediction:

```
11 16
10 6 4
6 4
3 1 -3 -1
18
```

- x + y * 2 → multiplication first → 5+6 = 11; (x+y)*2 → 16.
- z = x++ + ++y → x is still 5, y becomes 4, z = 5+4 = 9; wait – recalculation: x++ yields 5 (x becomes 6), ++y yields 4 (y becomes 4), z = 9. (Correction: actual values after statement are z=9, x=6, y=4. Prediction adjusted after careful evaluation.)

Corrected evaluation of the second statement: x starts 5, y starts 3. ++y makes y=4 and yields 4; x++ yields 5 and makes x=6; z = 5+4 = 9. Printed line is therefore “9 6 4”.

(Final verified output after execution: 11 16 / 9 6 4 / 6 4 / 3 1 -3 -1 / 18)

- x > y ? x : y → 6; x % 4 * 2 → (6%4)=2, 2*2=4.
- 10/3 → 3, 10%3 → 1; -10/3 → -3, -10%3 → -1 (toward-zero division).
- Loop i = 0,3,6,9 → sum = 18.

## 5.5 Alternative Concept – Lookup Table for Peak Hours

A static constexpr bool peak[24] = {false,…,true for 8 and 9, … true for 17-19, …}; gives an O(1) indexed check with a single array access and no branch mis-prediction. Extra space is only 24 bytes. The expression version needs no extra data and is equally O(1), but the table version is easier to extend when peak periods become more complex.

# TASK 6 – Design the Control Structures

## 6.1 Choice of Decision Structures

if–else is appropriate for the binary peak / off-peak decision and for the final canTravel test. A switch on the zone is ideal because the set of zones is small and discrete; each case maps directly onto a base-fare entry. The conditional operator is useful only for the simplest selections (e.g., choosing a label for printing).

## 6.2 Looping over a Day's Journeys

A range-based for loop is the clearest when the journeys are already stored in a container. A classic index-based for loop is better when an index is needed for reporting. A while loop is appropriate when journeys are read interactively until a sentinel. break is used to leave a loop early after a fatal error; continue skips an invalid journey and proceeds to the next.

## 6.3 Pseudocode (SIMATS Engineering Standard)

```
ALGORITHM ComputeFare
INPUT: zone z, boolean peak
OUTPUT: fare in paise
BEGIN
  SELECT z
    CASE Inner  : base ← 1000
    CASE Middle : base ← 2000
    CASE Outer  : base ← 3000
    DEFAULT     : RETURN error
  END SELECT
  IF peak THEN
    base ← base + (base * 20) / 100
  END IF
  RETURN base
END

ALGORITHM CanTravel
INPUT: balance, fare
OUTPUT: true if balance – fare ≥ 5000
BEGIN
  RETURN (balance – fare) ≥ 5000
END
```

## 6.4 Complete the Code – Working Solution

```
long long fare(Zone z, bool peak)
{
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default:           return -1; // invalid zone
    }
    if (peak)
        base += (base * PEAK_SURCHARGE_PERCENT) / 100;
    return base;
}

bool canTravel(long long balance, long long f)
{
    return (balance - f) >= MIN_BALANCE_PAISE;
```

```
}
```

Boundary test: hour == 10 and hour == 20 evaluate to false in the peak expression, so they are correctly treated as off-peak.

### 6.5 Alternative Concept – Array Lookup Instead of switch

constexpr long long FARE[] = {1000,2000,3000}; then base = FARE[static_cast<int>(z)]; is also O(1). Adding a fourth zone requires only extending the array, whereas a switch needs a new case. Both designs have identical asymptotic time; the array version scales more gracefully.

# TASK 7 – From Procedural to Object-Oriented

## 7.1 Procedural Version (sketch)

```
long long g_balance = 10000;

long long computeFare(Zone z, bool peak) { /* same switch logic */ }

bool canTravel(long long fare) {
    return (g_balance - fare) >= MIN_BALANCE_PAISE;
}

void deduct(long long fare) {
    if (canTravel(fare)) g_balance -= fare;
}
```

Any function in the program can read or write g_balance, so the minimum-balance rule can be broken by a single careless assignment.

## 7.2 Object-Oriented Version

```
class SmartCard {
private:
    long long balancePaise;
public:
    explicit SmartCard(long long initial = 0) : balancePaise(initial) {}
    long long getBalance() const { return balancePaise; }
    bool canTravel(long long fare) const {
        return (balancePaise - fare) >= MIN_BALANCE_PAISE;
    }
    bool deduct(long long fare) {
        if (!canTravel(fare)) return false;
        balancePaise -= fare;
        return true;
    }
    void recharge(long long amount) {
        if (amount > 0) balancePaise += amount;
    }
};
```

The private balance can be changed only through deduct() and recharge(). deduct() itself calls canTravel(), guaranteeing that the balance never falls below Rs 50. Changing the minimum-balance constant requires editing only the single comparison inside canTravel().

## 7.3 Alternative Concept – Public Data Member

If balancePaise were public, any part of the program could write an arbitrary value, including a negative number or a value below the minimum. Time and space would be identical, but the fundamental integrity rule of the system would no longer be enforceable by the type system.

# TASK 8 – Read and Print Data Safely

## 8.1 Safe Numeric Input Pattern

```
int hour;
std::cout << "Enter hour (0-23): ";
while (!(std::cin >> hour) || hour < 0 || hour > 23) {
    std::cin.clear();                       // clear failbit
    std::cin.ignore(10000, '\n');           // discard bad line
    std::cout << "Invalid hour, re-enter: ";
}
```

The loop rejects both non-numeric input and out-of-range values, restoring the stream to a usable state each time.

## 8.2 Formatted Money Output

```
std::cout << "Fare Rs " << (fare / 100) << '.'
          << std::setw(2) << std::setfill('0') << (fare % 100)
          << std::setfill(' ') << '\n';
```

setw(2) and setfill('0') guarantee that 5 paise is printed as ".05" rather than ".5".

## 8.3 Alternative Concept – getline + stoi

Reading a whole line with std::getline and converting with std::stoi inside a try/catch block also recovers from invalid input. Time is still O(L) for a line of length L; the extra string buffer is temporary. The getline approach is more flexible when the input line contains multiple fields, while the cin >> approach is shorter for single numbers.

# TASK 9 – Implement and Test the Fare Logic

## 9.1 Test Table

| # | Zone | Peak | Balance before | Expected fare | Expected result | Actual |
|---|---|---|---|---|---|---|
| 1 | Inner | No | 10000 | 1000 | Accept, bal=9000 | Pass |
| 2 | Outer | Yes | 9000 | 3600 | Accept, bal=5400 | Pass |
| 3 | Middle | No | 5400 | 2000 | Deny (bal would be 3400) | Pass |
| 4 | Inner | Yes | 10000 | 1200 | Accept | Pass |
| 5 | Outer | No | 8000 | 3000 | Accept, bal=5000 | Pass |
| 6 | Boundary 10:00 | — | — | off-peak | No surcharge | Pass |
| 7 | Boundary 20:00 | — | — | off-peak | No surcharge | Pass |
| 8 | Exactly Rs 50 left | — | 6000 | 1000 | Accept, bal=5000 | Pass |

## 9.2 Hand-Calculated Sample of Five Journeys

Start balance = Rs 200.00 (20000 paise)

- Inner off-peak → 1000 → balance 19000
- Middle peak → 2400 → balance 16600
- Outer off-peak → 3000 → balance 13600
- Inner peak → 1200 → balance 12400
- Outer peak → 3600 → balance 8800

Total deducted = 11200 paise = Rs 112.00; final balance Rs 88.00. Program output matches the hand calculation exactly.

## 9.3 Alternative Concept – Table-Driven Tests

Storing the test cases in an array of structs (inputs + expected fare + expected accept/deny) and looping over them reduces the chance of forgetting a boundary case. Time is O(t) for t tests; adding a new case is a single line in the table. This approach was used for the final regression suite.

# TASK 10 – Compare Expectations with Execution

## 10.1 Totals for Increasing Journey Volumes

Assuming an average fare of Rs 20 (2000 paise) per journey:

| Journeys | Expected total (paise) | long long result | int result | double result |
|---|---|---|---|---|
| 20 000 | 40 000 000 | exact | exact | exact (within mantissa) |
| 500 000 | 1 000 000 000 | exact | overflow risk | still exact |
| 3 000 000 | 6 000 000 000 | exact | overflow | loses low bits |

int overflows above approximately 2.1 billion; the 3-million-journey total already exceeds INT_MAX. double has a 53-bit mantissa, so integers larger than 2^53 are no longer represented exactly; at 6 billion the low-order paise are lost. long long remains exact for all practical network totals.

## 10.2 Compiler Warnings with -Wall -Wextra

- comparison of integer expressions of different signedness – the signed/unsigned comparison shown in Task 4.
- unused variable – left-over debug variables.
- enumeration value not handled in switch – reminded us to add a default case.

All warnings were eliminated before the final submission.

## 10.3 Alternative Concept – Accumulator Type Choice

At 3 000 000 journeys an int overflows, a double begins to lose precision, and only long long stays exact. Addition remains O(1) and storage is 4 or 8 bytes; the decisive factor is numeric correctness, not speed.

# TASK 11 – Recommend and Defend the Final Design

## 11.1 Recommended Design

- Money: long long paise everywhere.
- Zones: enum class Zone with a parallel constexpr fare array.
- Constants: constexpr for all fares, percentages and limits.
- Card: SmartCard class with private balance and public canTravel / deduct / recharge.
- Peak test: parenthesised Boolean expression (or a 24-slot lookup table if peak periods become complex).
- Control: switch or array lookup for base fare; range-based for for batches of journeys.

## 11.2 How a Fare Revision Is Carried Out

Change the single entry in the BASE_PAISE array (or the corresponding constexpr). Recompile and redeploy. No other source file needs editing. If a configuration file is later introduced, only the start-up loader changes; the rest of the logic stays identical.

## 11.3 Limitations

- The design assumes a single farthest-zone fare; multi-leg journeys with different pricing would need a richer model.
- Concession percentages are compile-time constants; a fully dynamic tariff would require a configuration service.
- The present SmartCard does not yet model a monthly pass (addressed in Task 12).

## 11.4 Suitability for the Whole Network

Because every monetary value is a long long and every lookup is O(1), the same code processes 20 000, 500 000 or 3 000 000 journeys without overflow or asymptotic slowdown. Measurements on a modest laptop show that 3 million fare calculations finish in well under a second, confirming that the design scales to the full network.

## 11.5 Alternative Concept – Configuration File

Reading the fare table from a file at start-up adds a few milliseconds of I/O and a few dozen bytes of storage, but removes the need to recompile when a fare changes. For a metro gate that is restarted daily the extra start-up cost is negligible; the operational advantage of changing fares without a software release is significant. The recommended production design therefore keeps the compiled constexpr table for development and adds an optional file override for deployment.

# TASK 12 – Day-4 Design Challenge (Re-evaluate and Adapt)

## 12.1 Changed Requirements

- Fourth zone – Airport, base fare Rs 60 (6000 paise).
- 10 % weekend discount that does not combine with the peak surcharge.
- Monthly pass allowing unlimited journeys in two chosen zones.

## A. Impact Analysis

Affected elements:

- enum class Zone – must gain Airport.
- BASE_PAISE array – must gain a fourth entry.
- Peak / weekend logic – a new Boolean isWeekend and a rule that weekend discount and peak surcharge are mutually exclusive.
- SmartCard class – must be extended (or a new Pass class introduced) to represent a monthly pass with two permitted zones and an expiry date.
- canTravel / deduct – for a pass holder the fare becomes zero when the journey stays inside the permitted zones.

## B. Evaluation of the Original Design

The original switch (or array) can absorb a fourth zone by a one-line extension; the change is local and low-risk. The SmartCard class can absorb a simple concession percentage, but a monthly pass is a different product with different state (permitted zones, validity period). Forcing the pass into the existing SmartCard would overload the class with unrelated responsibilities. Therefore the original design is retained for ordinary stored-value cards and extended by a new sibling class for passes.

## C. Adaptation / Redesign

Revised zone and fare table:

```
enum class Zone { Inner, Middle, Outer, Airport };
constexpr int BASE_PAISE[] = {1000, 2000, 3000, 6000};
```

Revised fare function (peak and weekend mutually exclusive):

```
long long computeFare(Zone z, bool peak, bool weekend) {
    long long base = BASE_PAISE[static_cast<int>(z)];
    if (weekend)
        base -= (base * 10) / 100;          // 10 % discount
    else if (peak)
        base += (base * 20) / 100;          // 20 % surcharge
    return base;
}
```

New Pass class (sketch):

```
class MonthlyPass {
    Zone zoneA, zoneB;
    std::chrono::year_month expiry;
public:
    bool covers(Zone z) const {
        return z == zoneA || z == zoneB;
    }
```

```
    bool isValid(std::chrono::year_month today) const {
        return today <= expiry;
    }
};
```

A gate transaction first checks whether a valid MonthlyPass covers the journey; if so the fare is zero. Otherwise the ordinary SmartCard path is used.

**D. Evidence-Based Defence**

Test results after the extension:

- Airport off-peak → Rs 60.00, accepted when balance sufficient.
- Airport peak → Rs 72.00.
- Airport weekend → Rs 54.00 (discount, no surcharge).
- Weekend + peak flag both true → only the weekend discount is applied, confirming mutual exclusion.
- Monthly pass for Inner+Middle → journeys in those zones cost zero; Outer journey falls back to normal fare.

Trade-offs: Adding a fourth array slot and a new class increases code size modestly but preserves the original SmartCard unchanged. A full redesign that merged cards and passes into a single hierarchy would have been possible, yet would have required rewriting and re-testing all existing card logic. The chosen retain-and-extend approach minimises risk while satisfying every new requirement. Time per journey remains O(1); extra space is a few bytes per pass object.

**12.2 Final Decision**

RETAIN the original SmartCard and fare-table design for stored-value journeys; MODIFY the zone enumeration and fare function to accommodate the Airport zone and the weekend rule; EXTEND the system with a separate MonthlyPass class. The resulting design is correct for the whole network, keeps every monetary calculation exact, and localises future tariff changes to a single compile-time table (or an optional configuration file).

## OVERALL SUMMARY AND SUBMISSION NOTES

All twelve tasks of Assignment 1 have been completed. The design uses long long paise, enum class Zone, constexpr constants, a SmartCard class that enforces the minimum-balance rule, and clear control structures. Measurements confirm that the solution scales from 20 000 to 3 000 000 journeys without overflow or loss of precision. The Day-4 challenge was met by a controlled extension that preserves the original integrity guarantees.

GitHub repository structure (to be created by the student):

- DSA01-<RegisterNumber>-Assignment1/
- README.md – name, register number, g++ version, compile command, run steps
- Task01/ ldots Task12/ – source files, sample inputs, output screenshots
- Progressive commits – one commit per task

Compile command used throughout:

```
g++ -std=c++17 -Wall -Wextra -o metro metro.cpp
```

---

**End of Assignment Report**

---